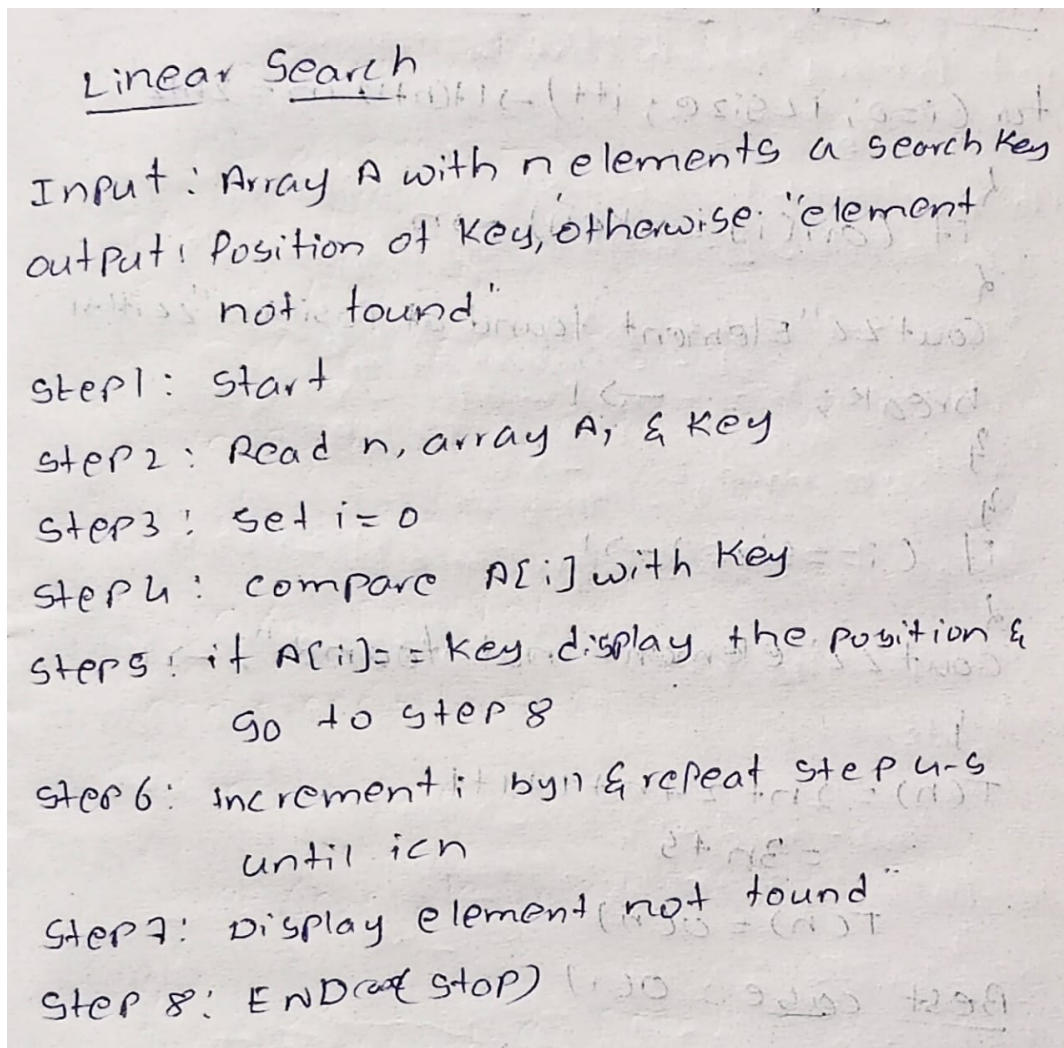


PRACTICAL-2

AIM: To implement and analyze the time complexity of Linear Search and Binary Search algorithms, and compare their searching efficiency for different input sizes.

LINEAR SEARCH:

ALGORITHM:



Linear Search

Input: Array A with n elements & search Key
Output: Position of Key, otherwise "element not found"

Step 1: Start
Step 2: Read n, array A, & Key
Step 3: Set i = 0
Step 4: Compare A[i] with Key
Step 5: if A[i] == Key, display the position & go to step 8
Step 6: increment i by 1 & repeat step 4-5 until i < n
Step 7: Display element "not found"
Step 8: END (stop)

PROGRAM/CODE :

```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
int size, arr[100], key, i;

cout << "Enter the size of the array: ";
cin >> size;

cout << "Enter " << size << " elements: ";
for (i = 0; i < size; i++)
{
    cin >> arr[i];
}

cout << "Enter the element to be searched: ";
cin >> key;

for (i = 0; i < size; i++)
{
    if (arr[i] == key)
    {
        cout << "Element found at position " << i + 1;
        break;
    }
}

if (i == size)
{
    cout << "Element not found";
}

return 0;
}
```

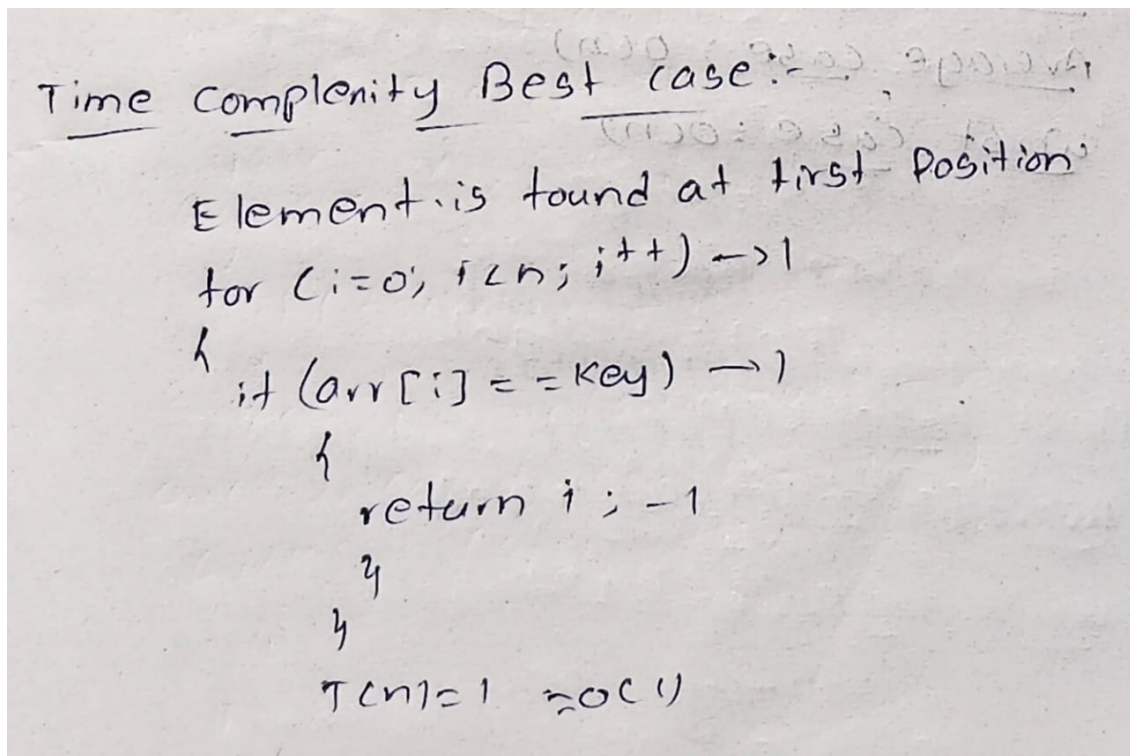
OUTPUT :

```
Enter the size of the array: 4
Enter 4 elements: 39
12
23
65
Enter the element to be searched: 12
Element found at position 2
```

TIME COMPLEXITY :

1.BEST CASE :

The best case occurs when the search element is found at the first position.



2.AVERAGE CASE :

The average case occurs when the element is found somewhere in the middle of the array.

3.WORST CASE :

The worst case occurs when the element is at the last position or is not present in the array.

Time complexity for linear search:

```
for (i=0; i < size; i++) → 1 (n+1) + n = 2n+2
{
    if (arr[i] == key) → n
    {
        cout << "Element found at position" << i+1
        break; → 1
    }
}
if (i == size) → 1
{
    cout << "element not found"; → 1
}
T(n) = 2n + 2 + n + 1 + 1 + 1
      = 3n + 5
      T(n) = O(n)
```

Best case : $O(1)$

Average case : $O(n)$

worst case : $O(n)$

BINARY SEARCH:

ALGORITHM:

Binary Search Algorithm

Input: sorted array A with n element & a search element

Output: Position of Key is found, otherwise 'element not found'

step 1: start

step 2: Read n sorted array A & Key

step 3: set low = 0 & high = n - 1

step 4: find mid = $(low + high) / 2$

step 5: if $A[mid] == Key$, display the Position

step 6: if $A[mid] < Key$, set low = mid + 1

step 7: if $A[mid] > Key$, set high = mid - 1

step 8: Repeat 4-7 while low $\leq$ high

step 9: if the element is not found, display 'element not found'

step 10: stop

PROGRAM/CODE :

```
#include <iostream>
using namespace std;

int BinarySearch(int arr[], int size, int search)
{
    int low = 0;
    int high = size - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;
```

```
if (search == arr[mid])
{
    return mid;
}
else if (search < arr[mid])
{
    high = mid - 1;
}
else
{
    low = mid + 1;
}
}

return -1;
}

int main()
{
    int size, arr[100], search, result;

    cout << "Enter the size of the array: ";
    cin >> size;

    cout << "Enter " << size << " elements in sorted order: ";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    cout << "Enter the element to be searched: ";
    cin >> search;

    result = BinarySearch(arr, size, search);

    if (result != -1)
    {
        cout << "Element found at position " << result + 1;
    }
    else
    {
        cout << "Element not found";
    }
}
```

```
return 0;  
}
```

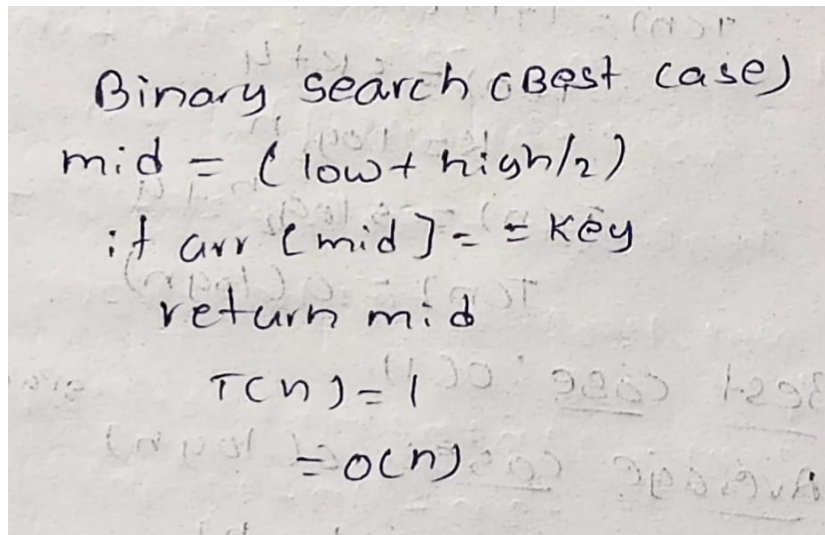
OUTPUT:

```
Enter the size of the array: 4  
Enter 4 elements in sorted order: 16  
26  
32  
45  
Enter the element to be searched: 26  
Element found at position 2
```

TIME COMPLEXITY:

1.BEST CASE :

The best case occurs when the search element is found at the middle position in the first comparison.



Binary search (Best case)
 $mid = (low + high / 2)$
if $arr[mid] == key$
return mid
 $TC(n) = 1$
 $= O(1)$

2.AVERAGE CASE :

The average case occurs when the element is found after several divisions of the search space.

3.WORST CASE:

The worst case occurs when the element is found after maximum divisions or is not present in the array.

Time Complexity for Binary Search

```

int BinarySearch (arr, size, search)
{
    low = 0;
    high = size - 1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (search == arr[mid])
        {
            return mid;
        }
        else if (search < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }
    return -1;
}

```

$T(n) = 1 + 1 + (K+1) + K + K + K + K + 1$
 $T(n) = 5K + 4$
 $K = \log_2 n$
 $T(n) = 5 \log_2 n + 4$
 $T(n) = O(\log n)$

Best case: $O(1)$ Space: $O(1)$
Average case: $O(\log n)$
Worst case: $O(\log n)$

CONCLUSION:

Thus, Linear Search and Binary Search were successfully implemented and their time complexities were analyzed. It was observed that Linear Search has a time complexity of $O(n)$, while Binary Search has $O(\log n)$, making Binary Search more efficient for searching in a sorted array.